EJERCICIOS PRÁCTICOS

UBER

Daniel Blanco Calviño

REQUISITOS

- **Requisitos funcionales.**
  - Los pasajeros pueden **solicitar viajes** indicando su ubicación actual y destino.
  - Los conductores deben **recibir solicitudes de viajes en tiempo real**.
  - **Seguimiento** del viaje, mostrando la ubicación del conductor y el progreso.
  - Cálculo de **tarifas dinámicas** en función de distancia, tiempo y demanda.
  - Registro de históricos de viajes para pasajeros y conductores.

- **Requisitos no funcionales.**
  - Alta disponibilidad.
  - Alta consistencia en asignación de conductores.
  - Se permite consistencia eventual en las ubicaciones.
  - Baja latencia.

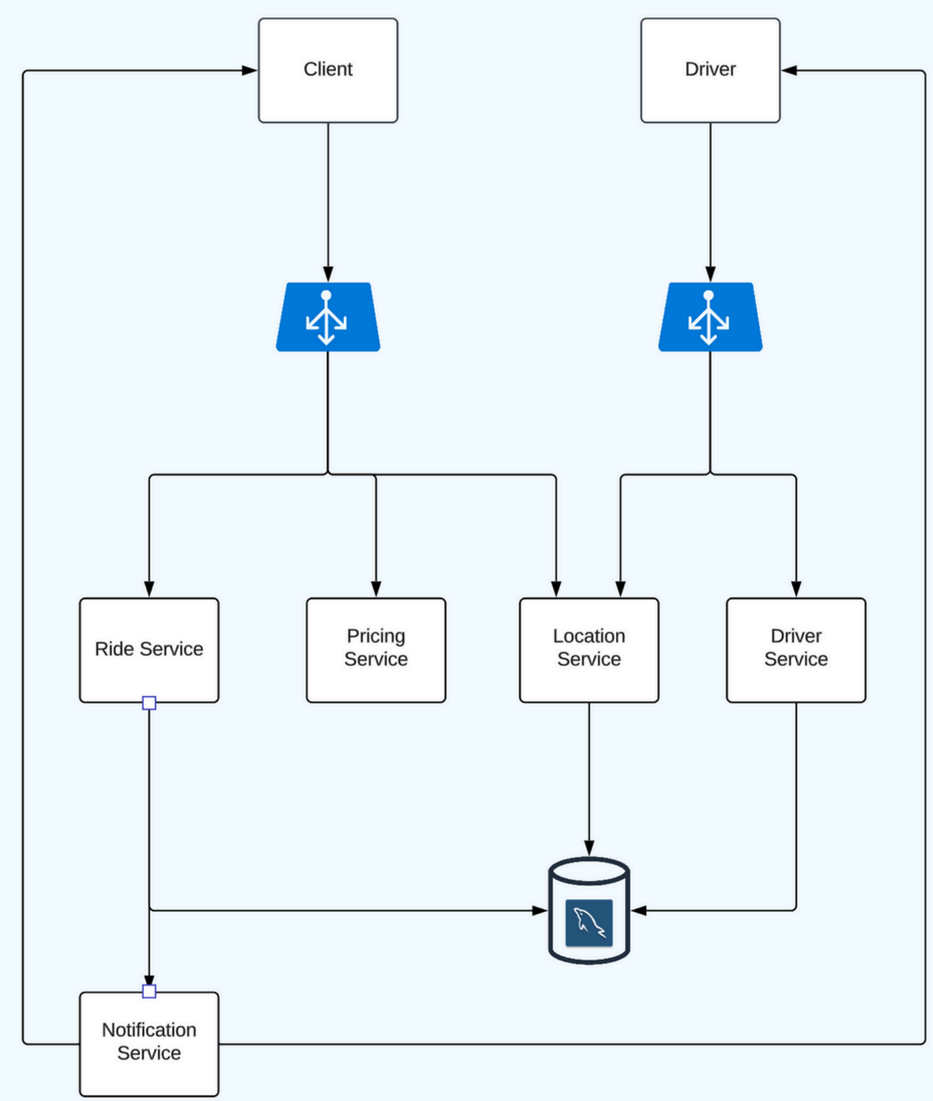
HIPÓTESIS Y ESTIMACIONES

- 150 millones de usuarios al mes.
  - 5M conductores que trabajan 6 horas de media al día.

- Dos viajes al mes de media por usuario.
  - 10 millones de viajes al día.
  - Los viajes tienen una duración media de 20 minutos.

- Consultas por segundo (QPS)
  - 10M viajes. Los usuarios consultan 10 veces la app: 100M / 3600 / 24 = 1150 QPS.
  - Cada conductor actualiza su ubicación en tiempo real cada 2 segundos.
    - 5M * 6 horas * actualización cada 2 segundos = 625.000 actualizaciones por segundo.

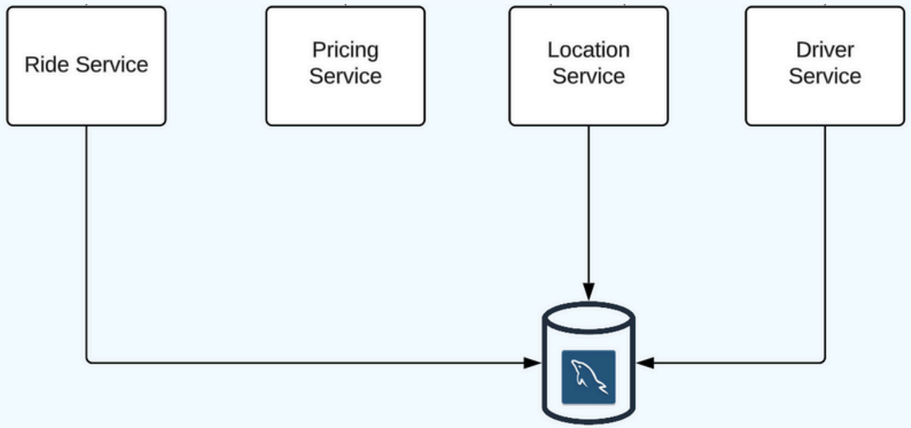
DISEÑO ALTO NIVEL



DISEÑO DETALLADO

- Problema: **gestionar datos geoespaciales**
  - Actualizaciones frecuentes de ubicación de los conductores.
  - Realizar búsquedas de proximidad eficientes.

- Base de Datos relacional.
  - Volumen de escrituras insostenible.
  - Búsquedas de proximidad muy ineficientes.
    - Posible solución: PostGIS.



DISEÑO DETALLADO

- **Solución: Redis.**
  - Comando GEOADD.
    - Almacenar ubicaciones.
  - Comando GEOSEARCH.
    - Búsquedas de proximidad.

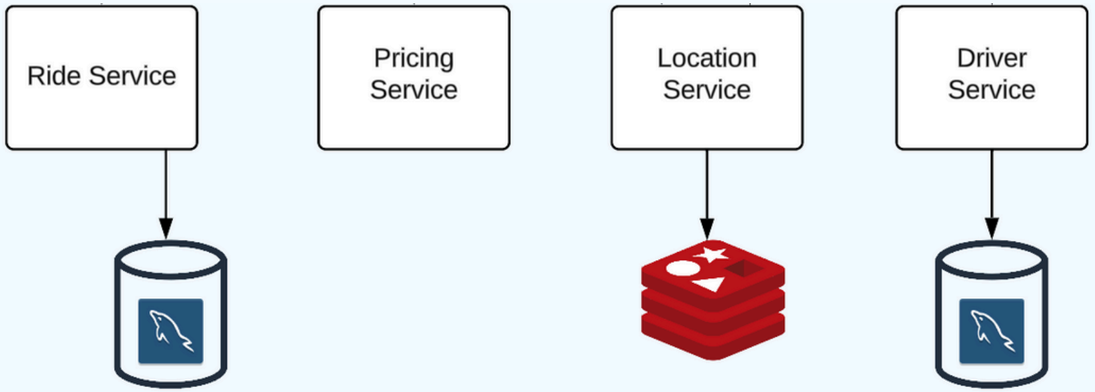
- Internamente se utiliza **geohashing**.
  - Se convierte pares latitud,longitud en cadenas de caracteres.
    - Se indexan y se almacenan en un conjunto ordenado.

DISEÑO DETALLADO

- Se almacena sólo la **ubicación más reciente** de cada conductor

- Se buscan conductores en un radio definido. Se puede aumentar si no se encuentran.
  - Lista con conductores válidos. Se selecciona por proximidad u otros criterios.

- **TTL** para eliminar registros antiguos de redis.

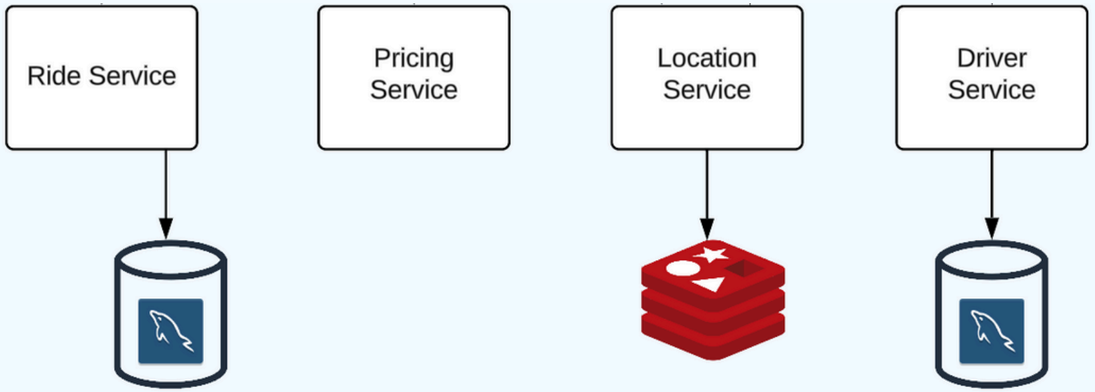


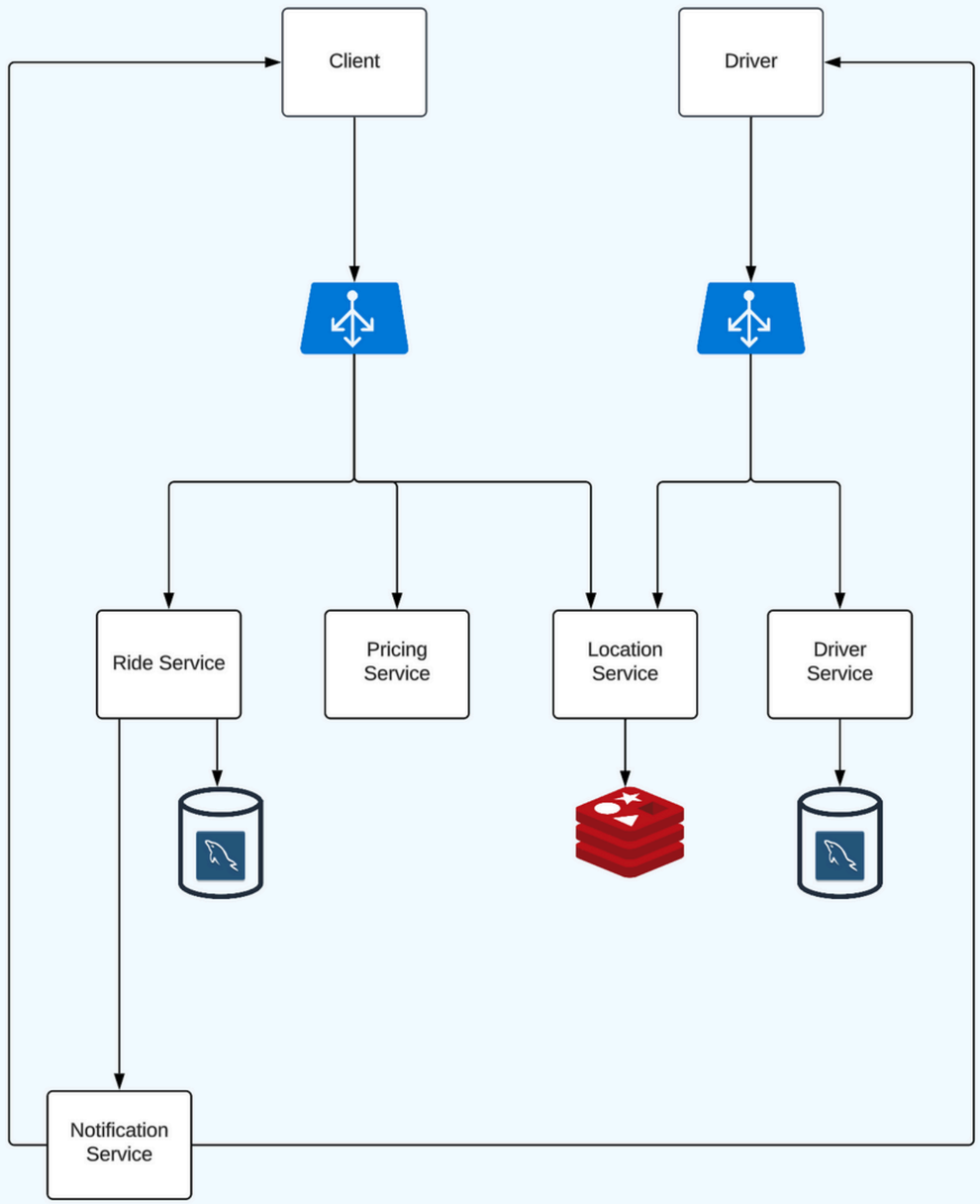
DISEÑO DETALLADO

- Almacén en **memoria**, riesgo de pérdida de datos.
  - Dependiendo de la frecuencia de actualización, **quizás no sea un problema.**
  - Considerar mitigar con persistencia en disco (**RDB**).

- Setup **multicluster** para aumentar disponibilidad y latencia.

- Implementar diferentes **frecuencias de actualización según actividad del conductor.**

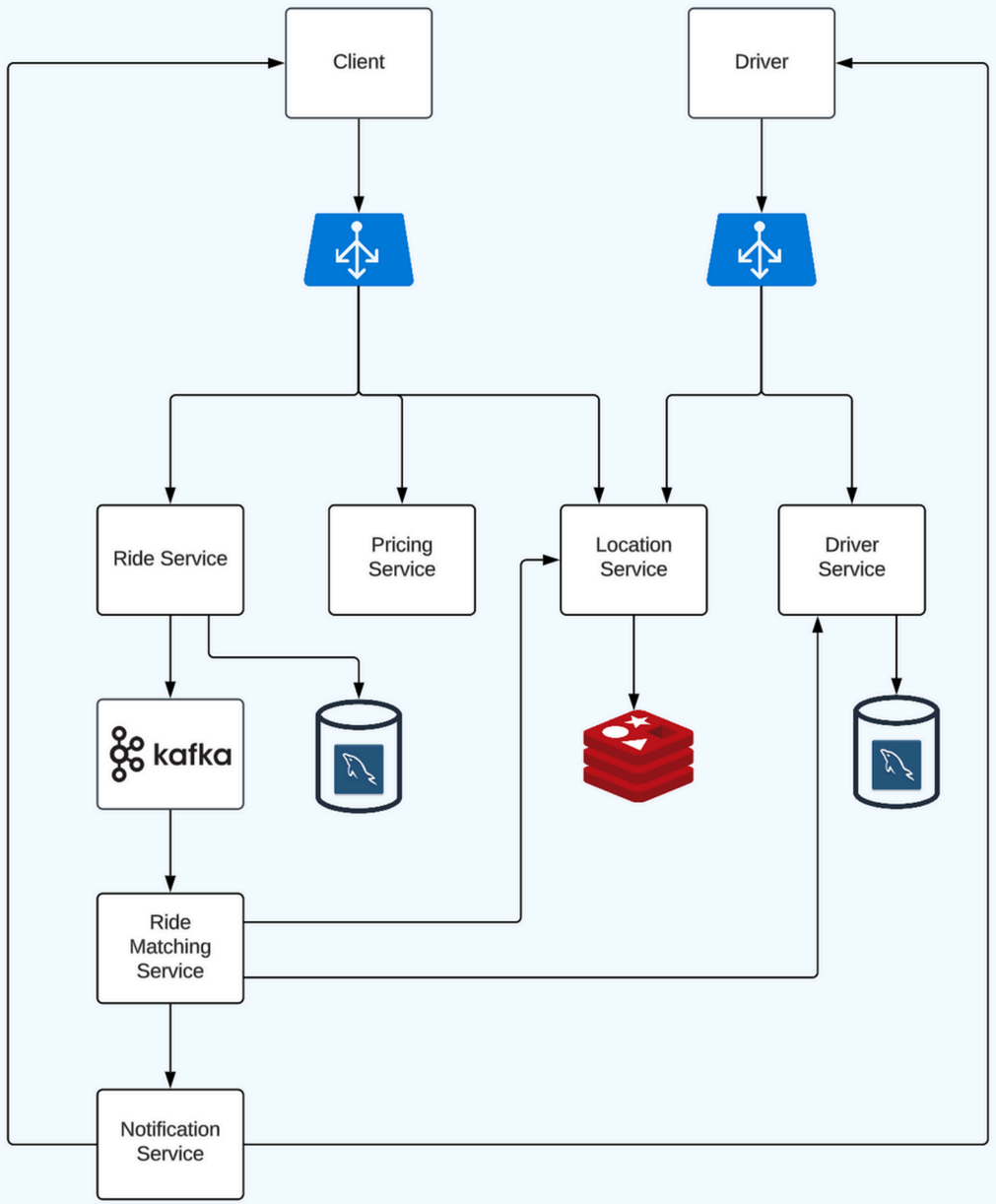




- Responsabilidades del Ride Service
  - Solicitudes de viajes.
  - Emparejamiento entre conductor y cliente.

- **Cada petición bloquea un hilo** hasta finalizar el emparejamiento.
  - Pueden ser segundos o incluso minutos.

- **No es tolerante a fallos.**


Client
Driver
Ride Service
Pricing Service
Location Service
Driver Service
kafka
Ride Matching Service
Notification Service

MÁS OPTIMIZACIONES

- Problema: Un conductor recibe múltiples solicitudes al mismo tiempo.
  - Muy **mala experiencia de usuario**. Distrae al conductor.
  - Hace el proceso de **matching muchísimo más lento**.

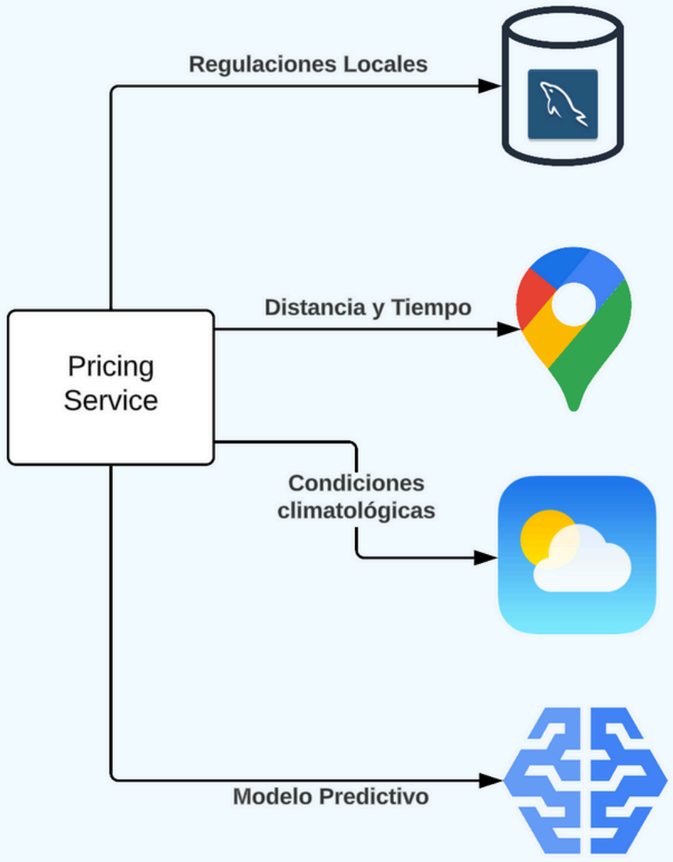
MÁS OPTIMIZACIONES

- Problema: Un conductor recibe múltiples solicitudes al mismo tiempo.
  - Muy **mala experiencia de usuario**. Distrae al conductor.
  - Hace el proceso de **matching muchísimo más lento**.

- Solución: **Bloqueos distribuidos** con redis.
  - Se inserta un lock con el ID del conductor al enviar la notificación.
    - **TTL pequeño**. Será el tiempo para responder.
  - Ride Matching Service sabrá qué conductores tienen notificaciones pendientes.
  - Si el conductor acepta el viaje, su estado pasa a no disponible.
  - Si lo rechaza, se elimina el lock.

MÁS OPTIMIZACIONES

- **Despliegue multidatacenter.**
  - Evitar replicar información innecesaria, como las ubicaciones.
    - Un usuario de EEUU no va a necesitar nunca la información de los conductores de España.
  - Replicar información crítica, como los perfiles de los usuarios y conductores.

MÁS OPTIMIZACIONES

- Tener en cuenta **regulaciones** locales.
  - Taxímetro o libertad de precios?
- Obtener **distancia y tiempo estimado de una API** de terceros.
- Enfoque reactivo.
  - Subir precios al detectar mayor demanda en una zona.
- Enfoque predictivo.
  - **Modelo de aprendizaje** con datos históricos.